# LangGraph 마스터 — 강사용 상세 대본

- 슬라이드 수: 324

- 부제: 상태 그래프와 운영형 에이전트 워크플로

- 기준: source-repo/03_langgraph/*.ipynb

---

### 01. AI 에이전트 개발 심화

안녕하세요. AI 에이전트 개발 심화 과정을 시작하겠습니다. 이번 트랙의 중심은 에이전트 실행을 상태, 노드, 엣지, 체크포인트, interrupt, time travel로 직접 통제하는 일입니다. 슬라이드는 개념의 흐름을 잡기 위한 자료이고, 뒤쪽 코드 부록은 실제 노트북의 실행 셀을 순서대로 확인하기 위한 자료입니다. 강의 중에는 API 이름을 외우는 것보다, 그 API가 어떤 문제를 해결하려고 등장했는지에 집중해 주세요. 이 기준을 잡으면 이후 실습에서 코드가 길어져도 실행 흐름을 놓치지 않습니다.

---

### 02. 오리엔테이션

이제 PART 01 오리엔테이션으로 넘어갑니다. 입문과 LangChain 기초에서 감춰져 있던 실행 흐름을 StateGraph와 Functional API로 직접 제어합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다.

---

### 03. LangGraph는 에이전트의 흐름을 코드가 아니라 그래프로 고정합니다

LangGraph는 에이전트의 흐름을 코드가 아니라 그래프로 고정합니다. 상태·노드·엣지를 명시하면 분기, 반복, 중단, 재개를 같은 모델로 설명합니다. 핵심 레이어는 다음과 같습니다. State는 공유 데이터에 해당합니다. 노드들이 읽고 갱신하는 TypedDict·MessagesState입니다. 리듀서가 누적 방식을 정합니다. 코드에서는 TypedDict · Annotated 같은 형태로 드러납니다. Node는 작업 단위에 해당합니다. 상태 일부를 받아 바뀐 부분만 반환하는 Python 함수입니다. LLM 호출도 하나의 노드가 됩니다. 코드에서는 def node(state) -> dict 같은 형태로 드러납니다. Edge는 흐름 제어에 해당합니다. 고정 연결과 조건부 연결로 다음 노드를 결정합니다. START와 END가 실행 경계를 만듭니다. 코드에서는 add_edge · add_conditional_edges 같은 형태로 드러납니다. 이 구조를 잡아 두면 뒤에서 API가 늘어나도 같은 기준으로 분류합니다.

---

### 04. 이번 트랙은 그래프 작성에서 운영 배포까지 한 번에 이어집니다

이번 트랙은 그래프 작성에서 운영 배포까지 한 번에 이어집니다. 각 노트북은 LangGraph가 제공하는 제어 지점을 하나씩 넓힙니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 GRAPH 단계에서는 StateGraph를 다룹니다. FUNC 단계에서는 entrypoint를 다룹니다. AGENT 단계에서는 ReAct를 다룹니다. CONTROL 단계에서는 interrupt를 다룹니다. RUN 단계에서는 server를 다룹니다. LangGraph 심화의 핵심은 모델이 아니라 상태 전이를 설계하는 능력입니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

---

### 05. 03_langgraph는 13개 노트북으로 그래프 제어의 전체 표면을 다룹니다

03_langgraph는 13개 노트북으로 그래프 제어의 전체 표면을 다룹니다. Graph API와 Functional API를 모두 써 보고, 운영에서 필요한 중단·재개·관측까지 확장합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 기본 구조는 01~03 소개·Graph API·Functional API입니다. 판단 기준은 StateGraph와 @entrypoint/@task를 모두 설명합니다. 패턴과 에이전트는 04~05 워크플로·ReAct 에이전트입니다. 판단 기준은 분기·병렬·

오케스트레이터·도구 루프를 직접 작성합니다. 상태 제어는 06~09 메모리·스트리밍·인터럽트·서브그래프입니다. 판단 기준은 실행 이력, 사람 승인, replay/fork, namespace 격리를 다룹니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

## 06. LangGraph 실행 모델

이제 PART 02 LangGraph 실행 모델으로 넘어갑니다. LangChain 위에서 복잡한 에이전트 흐름을 상태 그래프로 설계하는 이유를 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/01_introduction.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

## 07. LangGraph는 상태를 중심으로 실행·중단·재개를 설명합니다

LangGraph는 상태를 중심으로 실행·중단·재개를 설명합니다. 입문자는 노드보다 먼저 상태가 어떻게 변하는지 추적해야 합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 State는 그래프가 읽고 쓰는 공유 데이터입니다. 판단 기준은 TypedDict와 reducer를 구분합니다. Node는 상태를 받아 일부 상태를 반환하는 함수입니다. 판단 기준은 노드는 전체 상태가 아니라 변경분을 돌려줍니다. Edge는 다음 노드를 결정하는 연결입니다. 판단 기준은 조건부 엣지는 라우팅 함수와 함께 봅니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

## 08. Graph API와 Functional API는 표현 방식이 다를 뿐 같은 실행 모델을 공유합니다

Graph API와 Functional API는 표현 방식이 다를 뿐 같은 실행 모델을 공유합니다. 초반에 두 API의 역할을 구분하면 뒤의 노트북을 읽기 쉬워집니다. 카드별로 보면 Graph API는 상태 스키마, 노드, 엣지를 명시적으로 선언합니다. Functional API는 @entrypoint와 @task로 함수 호출처럼 그래프 기능을 사용합니다. 공통 기반은 둘 다 체크포인터, 스트리밍, interrupt 같은 런타임 기능을 사용합니다. 선택 기준은 흐름을 문서화해야 하면 Graph API, 코드 흐름이 자연스러우면 Functional API가 편합니다. 이 슬라이드는 선택지를 나열하는 장면입니다. 실제 설계에서는 지금 보이는 항목 중 무엇을 기본값으로 둘지, 무엇을 나중에 확장할지 결정해야 합니다.

---

## 09. LangChain·LangGraph·Deep Agents는 계층이 다르고 책임도 다릅니다

LangChain·LangGraph·Deep Agents는 계층이 다르고 책임도 다릅니다. LangGraph는 LangChain의 모델·도구 인터페이스 위에 상태 기반 오케스트레이션을 올립니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 LangChain은 모델과 도구를 연결하는 기본 인터페이스입니다. LangGraph는 상태·노드·엣지로 실행 흐름을 직접 제어합니다. Deep Agents는 LangGraph 위에 파일·서브에이전트·메모리·스킬을 얹고, 필요하면 TodoListMiddleware로 계획을 추가하는 하네스입니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

---

## 10. Graph API와 Functional API는 같은 런타임을 다른 표면으로 다룹니다

Graph API와 Functional API는 같은 런타임을 다른 표면으로 다룹니다. 복잡한 흐름은 Graph API, 직선형 파이프라인은 Functional API가 읽기 쉽습니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Graph API는 StateGraph에 노드와 엣지를 명시입니다. 판단 기준은 분기·반복·시각화·복잡한 에이전트 제어입니다. Functional API는 @entrypoint와 @task로 함수형 작성입니다. 판단 기준은 작업 파이프라인·병렬 태스크·짧은 워크플로입니다. 공통점은 checkpointer와 stream을 공유입니다. 판단 기준은 중단·재개·관측을 같은 방식으로 다룸입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 11. StateGraph의 최소 실행 흐름은 생성·노드·엣지·컴파일·호출입니다

StateGraph의 최소 실행 흐름은 생성·노드·엣지·컴파일·호출입니다. 노드는 전체 상태가 아니라 바뀐 필드만 dict로 반환합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 StateGraph는 상태 스키마를 기준으로 그래프 빌더를 만듭니다. START/END는 실행의 시작점과 종료점을 명시합니다. compile는 연결 오류를 검증하고 실행 가능한 그래프로 바꿉니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 typing에서 TypedDict; graph에서 StateGraph, START, END를 준비합니다. 클래스 State는 데이터 구조나 정책 단위를 정의합니다. 함수 count_words는 실제 작업 단위입니다. 노트북 기준 위치는 01_introduction.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 12. 처음에는 그래프를 '제어 가능한 실행 로그'로 보는 편이 쉽습니다

처음에는 그래프를 '제어 가능한 실행 로그'로 보는 편이 쉽습니다. 그래프는 그림이 아니라 실행 중 상태가 어디서 바뀌었는지 추적하는 구조입니다. 묶음별로 읽으면 상태가 명시되면 테스트가 쉬워집니다 여기서는 각 노드는 입력 상태와 출력 업데이트만 검증하면 됩니다; 조건부 엣지는 라우팅 함수 하나로 분리됩니다를 봅니다. 컴파일된 그래프는 LangGraph 런타임 기능을 얻습니다 여기서는 stream, checkpointer, interrupt, get_state_history를 같은 방식으로 사용합니다; Studio와 서버 배포에서도 같은 그래프 객체를 대상으로 삼습니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 13. Graph API

이제 PART 03 Graph API로 넘어갑니다. StateGraph의 기본 문법과 add_node 고급 옵션, Send·Command를 한 번에 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/02_graph_api.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 14. 조건부 엣지는 상태 값을 다음 노드 이름으로 바꾸는 라우터입니다

조건부 엣지는 상태 값을 다음 노드 이름으로 바꾸는 라우터입니다. 라우팅 함수는 비즈니스 분류 결과를 그래프 제어 흐름으로 바꿉니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 classify는 입력 질문을 category로 분류합니다. route는 category 값을 노드 이름으로 반환합니다. add_conditional_edges는 분기 가능한 목적지를 그래프에 등록합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 typing에서 Literal를 준비합니다. 함수 route는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node, add_edge, add_conditional_edges입니다. 노트북 기준 위치는 02_graph_api.ipynb conditional edges입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 15. 입출력 스키마를 분리하면 내부 상태를 외부 응답에서 숨길 수 있습니다

입출력 스키마를 분리하면 내부 상태를 외부 응답에서 숨길 수 있습니다. 프로덕션 API에서는 내부 중간값과 외부 응답 형식을 분리해야 합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 InputSchema는 호출자가 넣는 입력만 정의합니다. InternalState는 그래프 내부 계산에 필요한 중간 필드를 포함합니다. OutputSchema는 최종 사용자에게 돌려줄 필드만 노출합니다. 코드는 다음 지점을 중심으로 읽습니다. 클래스 InputSchema, OutputSchema, InternalState는 데이터 구조나 정책 단위를 정의합니다. 핵심 API는 StateGraph입니다. 노트북 기준 위치는 02_graph_api.ipynb input/output schema입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 16. 조건부 엣지는 그래프를 if문 더미가 아니라 라우팅 표면으로 만듭니다

조건부 엣지는 그래프를 if문 더미가 아니라 라우팅 표면으로 만듭니다. 분기 로직이 함수로 분리되면 테스트와 관측이 쉬워집니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 분류 노드는 state를 읽고 category 같은 라우팅 키를 만듭니다. route 함수는 다음 노드 이름을 Literal로 반환합니다. 핸들러 노드는 각 경로는 독립적으로 테스트합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

## 17. Annotated 리듀서는 상태를 덮어쓸지 누적할지 결정합니다

Annotated 리듀서는 상태를 덮어쓸지 누적할지 결정합니다. 노드가 같은 키를 갱신해도 리듀서가 있으면 리스트처럼 안전하게 합쳐집니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 기본값은 리듀서가 없으면 마지막 업데이트가 값을 덮어씁니다. operator.add는 리스트를 누적할 때 가장 자주 쓰는 패턴입니다. 메시지는 MessagesState는 add_messages 리듀서를 미리 제공합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 typing에서 Annotated, TypedDict; operator를 준비합니다. 클래스 AccState는 데이터 구조나 정책 단위를 정의합니다. 함수 step_a, step_b는 실제 작업 단위입니다. 노트북 기준 위치는 02_graph_api.ipynb Reducer입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 18. Graph API의 운영 옵션은 노드 선언 시점에 붙입니다

Graph API의 운영 옵션은 노드 선언 시점에 붙입니다. retry, timeout, cache, error_handler, defer는 실행 정책이지 노드 내부 로직이 아닙니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 retry_policy는 일시 장애 자동 재시도입니다. 판단 기준은 네트워크·일시적 API 오류입니다. timeout는 노드 실행 시간 제한입니다. 판단 기준은 외부 호출이 긴 노드입니다. cache_policy는 같은 입력 결과 재사용입니다. 판단 기준은 비싼 계산·검색입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 19. Send는 동적 fan-out을 만들고 Command는 상태 업데이트와 라우팅을 함께 표현합니다

Send는 동적 fan-out을 만들고 Command는 상태 업데이트와 라우팅을 함께 표현합니다. map-reduce처럼 입력 개수에 따라 worker 수가 달라지는 흐름에서 Send가 핵심입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 Send는 같은 노드를 여러 입력으로 동적으로 호출합니다. Command는 update와 goto를 한 객체에 담아 라우팅합니다. 부모 그래프는 Command.PARENT로 서브그래프 밖 라우팅도 표현합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 typing에서 Annotated, TypedDict; types에서 Send를 준비합니다. 클래스 FanOutState는 데이터 구조나 정책 단위를 정의합니다. 함수 dispatch, worker는 실제 작업 단위입니다. 노트북 기준 위치는 02_graph_api.ipynb Send입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 20. Functional API

이제 PART 04 Functional API로 넘어갑니다. @entrypoint와 @task로 병렬 작업, previous 메모리, 캐싱, 구조화 interrupt를 다룹니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/03_functional_api.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 21. @task 결과는 future로 돌아오고 result()를 호출할 때 값이 필요해집니다

@task 결과는 future로 돌아오고 result()를 호출할 때 값이 필요해집니다. Functional API의 병렬성은 함수처럼 보이지만 즉시 값이 아니라 future를 다룬다는 점에서 출발합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 @task는 재시도와 체크포인트가 가능한 작업 단위입니다. future는 task 호출 결과는 나중에 result()로 수집합니다. @entrypoint는 전체 실행의 진입점과 체크포인터를 정의합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 fetch_data, process_data, data_pipeline는 실제 작업 단위입니다. 노트북 기준 위치는 03_functional_api.ipynb @task + @entrypoint입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 22. Functional API는 편하지만 결정론성 요구사항을 지켜야 합니다

Functional API는 편하지만 결정론성 요구사항을 지켜야 합니다. 재실행 가능한 그래프는 같은 입력과 체크포인트에서 같은 흐름으로 돌아와야 합니다. 묶음별로 읽으면 지켜야 할 것에서는 entrypoint 안에서 외부 부작용을 무작정 실행하지 않습니다; 오래 걸리거나 실패가능한 작업은 @task로 분리합니다; 체크포인터를 쓸 때 thread_id를 명시합니다를 봅니다. 헷갈리기 쉬운 것에서는 print나 임시 변수는 저장되지 않습니다; entrypoint.final은 반환값과 저장값을 분리할 때 사용합니다; previous는 같은 thread에서 이전 저장값을 읽을 때 사용합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 23. @task는 체크포인트되는 비동기 작업 단위입니다

@task는 체크포인트되는 비동기 작업 단위입니다. task 호출은 Future를 반환하고 .result()로 결과를 수집합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 @task는 재시도·타임아웃·캐싱을 붙일 수 있는 작업 단위입니다. @entrypoint는 워크플로 진입점이며 checkpointer를 받습니다. thread_id는 같은 entrypoint라도 실행 이력을 구분합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 fetch_data, process_data, data_pipeline는 실제 작업 단위입니다. 노트북 기준 위치는 03_functional_api.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 24. Functional API의 핵심은 병렬 시작과 결정론적 재실행입니다

Functional API의 핵심은 병렬 시작과 결정론적 재실행입니다. 비결정적 작업은 task 안으로 넣어야 checkpoint 재개 시 결과가 흔들리지 않습니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 병렬 실행은 여러 task를 먼저 호출 후 result 수집입니다. 판단 기준은 독립 작업을 동시에 시작입니다. previous는 entrypoint 인자 previous입니다. 판단 기준은 같은 thread의 이전 저장값 접근입니다. entrypoint.final은 value와 save 분리입니다. 판단 기준은 사용자 반환값과 다음 실행 저장값 분리입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 25. previous와 entrypoint.final은 사용자 반환값과 저장값을 분리합니다

previous와 entrypoint.final은 사용자 반환값과 저장값을 분리합니다. 보이는 답변은 문자열이어도 다음 실행에는 숫자 상태를 저장합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 previous는 이전 실행의 save 값이 자동 주입됩니다. value는 이번 호출자가 받는 반환

값입니다. save는 다음 호출의 previous로 저장되는 값입니다. 코드는 다음 지점을 중심으로 읽습니다. 데코레이터 entrypoint, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 counter, smart_counter는 실제 작업 단위입니다. 핵심 API는 InMemorySaver입니다. 두 코드 블록을 비교하면 단기 메모리 블록은 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 반환/저장 분리 블록은 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 26. Functional API에서도 interrupt는 Command(resume=...)으로 재개합니다

Functional API에서도 interrupt는 Command(resume=...)으로 재개합니다. 사람 검토가 들어가는 작업은 checkpointer가 있어야 중단 지점을 기억합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 interrupt는 사람에게 보여줄 질문과 검토 대상을 구조화합니다. Command는 resume 값으로 중단 지점에 답을 전달합니다. 재개는 같은 thread_id를 써야 같은 실행으로 이어집니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 types에서 Command, interrupt를 준비합니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 review, approval_flow는 실제 작업 단위입니다. 노트북 기준 위치는 03_functional_api.ipynb interrupt입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 27. 워크플로 패턴

이제 PART 05 워크플로 패턴으로 넘어갑니다. Prompt chaining, parallelization, routing, orchestrator-worker, evaluator-optimizer를 StateGraph로 구현합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/04_workflows.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

### 28. Send는 실행 시점에 워커 수가 정해지는 fan-out 패턴에 사용합니다

Send는 실행 시점에 워커 수가 정해지는 fan-out 패턴에 사용합니다. 고정된 분기보다 동적으로 여러 작업을 만들어야 할 때 Send가 필요합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 dispatch는 입력 리스트를 worker 호출 목록으로 바꿉니다. worker은 각 항목을 독립적으로 처리합니다. reducer은 Annotated reducer로 결과를 모읍니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 types에서 Send를 준비합니다. 함수 dispatch, worker는 실제 작업 단위입니다. 핵심 API는 add_edge, add_conditional_edges, Send입니다. 노트북 기준 위치는 04_workflows.ipynb orchestrator-worker입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 29. 워크플로 패턴은 실패 지점과 비용 구조가 서로 다릅니다

워크플로 패턴은 실패 지점과 비용 구조가 서로 다릅니다. 패턴 이름보다 어떤 운영 문제가 생기는지 기준으로 선택해야 합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Prompt Chaining는 검토 단계를 순서대로 통과하는 작업입니다. 판단 기준은 앞 단계 오류가 뒤로 전파됩니다. Parallelization은 서로 독립적인 분석 작업입니다. 판단 기준은 결과 합성 기준이 필요합니다. Routing는 질문 유형별 처리 분기입니다. 판단 기준은 분류 오류가 곧 잘못된 경로가 됩니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 30. 다섯 패턴은 'LLM을 몇 번 부르나'보다 제어 구조가 다릅니다

다섯 패턴은 'LLM을 몇 번 부르나'보다 제어 구조가 다릅니다. 각 패턴은 실패 지점과 테스트 방식이 달라집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Prompt Chaining는 A 결과를 B 입력으로 전달입니다. 판단 기준은 단계별 품질 게이트가 필요한 글쓰기입니다. Parallelization은 독립 작업 동시 실행 후 합치기입니다. 판단 기준은 요약·분류·검토를 병렬로 처리입니다. Routing는 분류 결과로 다음 노드 선택입니다. 판단 기준은 문의 유형별 전문 처리입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 31. Orchestrator-Worker는 계획 결과만큼 worker를 동적으로 만듭니다

Orchestrator-Worker는 계획 결과만큼 worker를 동적으로 만듭니다. StateGraph에서는 Send()가 이 동적 fan-out의 연결점입니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Orchestrator은 전체 작업을 보고 worker 입력 목록을 만듭니다. Worker은 각 작업 단위를 독립적으로 처리합니다. Reducer은 worker 결과를 리스트나 문서로 합칩니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

## 32. Routing은 조건부 엣지 하나로 분류와 처리를 분리합니다

Routing은 조건부 엣지 하나로 분류와 처리를 분리합니다. 분류 노드를 테스트하면 라우팅 품질을 별도 지표로 봅니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 classify는 입력 질문을 category로 바꿉니다. route는 category를 다음 노드 이름으로 변환합니다. handler은 각 경로의 처리는 독립 함수가 맡습니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 typing에서 Literal, TypedDict를 준비합니다. 클래스 RouteState는 데이터 구조나 정책 단위를 정의합니다. 함수 classify, route는 실제 작업 단위입니다. 노트북 기준 위치는 04_workflows.ipynb Routing입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 33. Evaluator-Optimizer는 품질 기준을 그래프의 반복 조건으로 만듭니다

Evaluator-Optimizer는 품질 기준을 그래프의 반복 조건으로 만듭니다. 평가 결과가 통과일 때만 END로 가고, 아니면 optimizer로 되돌립니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 GENERATE 단계에서는 초안 생성를 다룹니다. EVALUATE 단계에서는 점수화를 다룹니다. ROUTE 단계에서는 통과 판단를 다룹니다. OPTIMIZE 단계에서는 수정를 다룹니다. 이 패턴은 LLM을 믿는 대신 검사 가능한 품질 루프를 그래프에 넣는 방식입니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

## 34. ReAct 에이전트

이제 PART 06 ReAct 에이전트로 넘어갑니다. Graph API와 Functional API로 도구 호출 루프를 직접 만들고 스트리밍과 메모리까지 연결합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/05_agents.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 35. Graph API 에이전트는 모델 호출과 도구 실행을 명시적으로 나눕니다

Graph API 에이전트는 모델 호출과 도구 실행을 명시적으로 나눕니다. ReAct를 그래프로 그리면 모델이 언제 도구를 부르고 언제 종료하는지 보입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 call_model은 메시지를 보고 다음 AIMessage를 생성합니다. 판단 기준은 model.bind_tools(tools)입니다. ToolNode는

AIMessage의 tool_calls를 실행합니다. 판단 기준은 ToolNode(tools)입니다. tools_condition은 도구 호출 여부에 따라 분기합니다. 판단 기준은 tools 또는 END입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

### 36. Functional API 에이전트는 while 루프로 ReAct를 직접 표현합니다

Functional API 에이전트는 while 루프로 ReAct를 직접 표현합니다. 그래프 구조보다 코드 흐름을 읽는 것이 쉬운 팀에서는 Functional API 구현이 더 직관적일 수 있습니다. 묶음별로 읽으면 반복 루프에서는 모델을 호출합니다; tool_calls가 없으면 최종 답변으로 종료합니다; tool_calls가 있으면 도구를 실행하고 메시지에 추가합니다를 봅니다. 메모리 연결에서는 checkpointer가 있으면 같은 thread_id에서 대화가 이어집니다; thread_id가 다르면 대화 상태가 분리됩니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

---

### 37. ReAct 루프는 llm 노드와 tools 노드의 조건부 반복입니다

ReAct 루프는 llm 노드와 tools 노드의 조건부 반복입니다. create_agent가 감추던 흐름을 StateGraph로 풀어 쓰면 디버깅 지점이 분명해집니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 llm은 도구 호출 여부를 판단하고 AIMessage를 생성합니다. tools는 tool_calls를 실행해 ToolMessage를 만듭니다. should_continue는 tool_calls가 있으면 tools, 없으면 END로 보냅니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

---

### 38. bind_tools()는 모델이 호출할 수 있는 도구 스키마를 모델에 붙입니다

bind_tools()는 모델이 호출 가능한 도구 스키마를 모델에 붙입니다. 도구 함수 자체는 일반 Python 함수지만 모델에는 이름·설명·입력 스키마로 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 @tool은 함수명과 docstring을 도구 설명으로 바꿉니다. tools_by_name는 tool_call name으로 실제 함수를 찾기 위한 매핑입니다. bind_tools는 모델 호출 결과에 tool_calls가 생길 수 있게 합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add, multiply는 실제 작업 단위입니다. 노트북 기준 위치는 05_agents.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 39. StateGraph ReAct 루프는 조건부 엣지 하나로 반복 여부를 결정합니다

StateGraph ReAct 루프는 조건부 엣지 하나로 반복 여부를 결정합니다. 마지막 AIMessage에 tool_calls가 있으면 도구 노드로, 없으면 종료로 갑니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 messages reducer은 Annotated와 operator.add 또는 MessagesState로 메시지를 누적합니다. ToolMessage는 각 tool_call_id와 결과를 정확히 매칭해야 합니다. checkpointer은 메모리 있는 에이전트는 compile(checkpointer=...)로 만듭니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 typing에서 Literal; graph에서 END를 준비합니다. 함수 should_continue는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, compile, add_node, add_edge, add_conditional_edges입니다. 노트북 기준 위치는 05_agents.ipynb Graph API ReAct입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 40. Graph API와 Functional API 에이전트는 제어 표면이 다릅니다

Graph API와 Functional API 에이전트는 제어 표면이 다릅니다. 같은 ReAct라도 어디에 복잡성이 드러나는지가 다릅니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 흐름 표현은 노드·엣지로 시각화입니다. 판단 기준은 while 루프와 task 호출입니다. 디버깅은 노드별 업데이트가 명확입니다. 판단 기준은 함수 호출 흐름이 읽기 쉬움입니다. 복잡한 분기는 조건부 엣지로 확장 용이입니다. 판단 기준은 분기가 많으면 코드가 길어짐입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 41. 지속성과 메모리

이제 PART 07 지속성과 메모리로 넘어갑니다. checkpointer, get_state, get_state_history, update_state, InMemoryStore, trimming을 묶어 봅니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/06_persistence_and_memory.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 42. get_state와 get_state_history는 실행 상태를 운영자가 조회하는 표면입니다

get_state와 get_state_history는 실행 상태를 운영자가 조회하는 표면입니다. 체크포인터를 쓰면 현재 상태와 과거 실행 이력을 코드로 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 get_state는 현재 체크포인트 상태를 조회합니다. get_state_history는 노드별 실행 이력을 확인합니다. config는 thread_id가 조회 대상 실행을 정합니다. 코드는 다음 지점을 중심으로 읽습니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 06_persistence_and_memory.ipynb state inspection입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 43. update_state는 저장된 실행 상태를 외부에서 고치는 운영 도구입니다

update_state는 저장된 실행 상태를 외부에서 고치는 운영 도구입니다. 오답 수정, 사람 검토, time travel은 저장 상태를 수정 가능하다는 전제에서 작동합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 as_node는 어떤 노드가 수정한 것처럼 기록할지 정합니다. values는 덮어쓸 상태 변경분입니다. thread_id는 수정 대상 실행을 지정합니다. 코드는 다음 지점을 중심으로 읽습니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 06_persistence_and_memory.ipynb update_state입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 44. checkpointer를 붙이면 각 단계의 상태가 자동 저장됩니다

checkpointer를 붙이면 각 단계의 상태가 자동 저장됩니다. thread_id가 같으면 같은 실행 이력을 이어가고, 다르면 완전히 독립된 상태가 됩니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 InMemorySaver은 실습용 체크포인터입니다. 프로세스 종료 시 사라집니다. thread_id는 대화 또는 실행 세션을 구분합니다. get_state는 현재 저장된 마지막 상태를 조회합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 memory에서 InMemorySaver; graph에서 MessagesState를 준비합니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 06_persistence_and_memory.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 45. 저장된 상태는 조회·이력 확인·수정까지 가능합니다

저장된 상태는 조회·이력 확인·수정까지 가능합니다. 운영에서는 실패 분석과 수동 보정의 기본 도구가 됩니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 get_state(config)는 현재 체크포인트 상태입니다. 판단 기준은 마지막 실행 결과 확인입니다. get_state_history(config)는 체크포인트 목록입니다. 판단 기준은 어느 노드에서 상태가 바뀌었는지 추적입니다. update_state(config, values)는 새 체크포인트입니다. 판단 기준은 사람이 상태를 고친 뒤 재개입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 46. InMemoryStore는 thread를 넘어 공유되는 장기 메모리를 담당합니다

InMemoryStore는 thread를 넘어 공유되는 장기 메모리를 담당합니다. LangGraph 1.x 권장 패턴은 context_schema와 Runtime[Context]를 함께 쓰는 것입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 store는 사용자 선호 같은 장기 정보를 namespace별로 저장합니다. context는 user_id를 state가 아니라 호출 컨텍스트로 주입합니다. Runtime는 runtime.context와 runtime.store로 타입 안전하게 접근합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 dataclasses에서 dataclass; runtime에서 Runtime를 준비합니다. 클래스 Context는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. 노트북 기준 위치는 06_persistence_and_memory.ipynb Runtime + Store입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 47. 긴 대화는 저장만큼 삭제와 압축 전략이 중요합니다

긴 대화는 저장만큼 삭제와 압축 전략이 중요합니다. 메시지를 무한히 쌓으면 비용과 품질이 동시에 나빠집니다. 묶음별로 읽으면 trim_messages는 모델 호출 직전에 컨텍스트를 줄입니다. 여기서는 system 메시지는 유지하고 최근 메시지를 우선 남기는 전략을 자주 씁니다; 토큰 예산을 기준으로 오래된 메시지를 잘라냅니다를 봅니다. RemoveMessage는 저장된 메시지를 명시적으로 제거합니다. 여기서는 실행 이력은 남기되 모델 입력에는 불필요한 과거를 줄입니다; 요약 메모리와 함께 쓰면 장기 대화를 더 안정적으로 운영합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 48. 스트리밍

이제 PART 08 스트리밍으로 넘어갑니다. values, updates, messages, custom, v2 StreamPart, nostream, Event Streaming v3를 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/07_streaming.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 49. stream_mode는 누구에게 어떤 실행 정보를 보여줄지 정하는 선택입니다

stream_mode는 누구에게 어떤 실행 정보를 보여줄지 정하는 선택입니다. 사용자 UI와 운영 로그는 필요한 스트림 종류가 다릅니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 values는 각 단계 이후 전체 상태입니다. 판단 기준은 디버깅과 상태 스냅샷입니다. updates는 노드별 변경분입니다. 판단 기준은 운영 로그와 진행 표시입니다. messages는 LLM 토큰 스트림입니다. 판단 기준은 채팅 UI입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 50. custom 스트리밍은 그래프 내부 진행률을 사용자 언어로 내보낼 때 사용합니다

custom 스트리밍은 그래프 내부 진행률을 사용자 언어로 내보낼 때 사용합니다. 토큰만 보여주면 장기 작업의 실제 진행 상태를 알기 어렵습니다. 묶음별로 읽으면 사용 시점에서는 파일 처리, 검색, 분석처럼 중간 단계가 긴 작업입니다; 사용자에게 현재 단계와 남은 작업을 설명해야 합니다를 봅니다. 설계 기준에서는 노드 이름 대신 사용자에게 의미 있는 이벤트명을 씁니다; 성공 이벤트와 실패 이벤트를 구분합니다; 프런트엔드에서 표시할 최소 필드를 정합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 51. stream_mode는 '무엇을 밖으로 보여줄지'의 선택입니다

stream_mode는 '무엇을 밖으로 보여줄지'의 선택입니다. 최종 답만 필요한지, 노드 진행률이 필요한지, 토큰이 필요한지에 따라 모드가 달라집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 values는 전체 상태 스냅샷입니다. 판단 기준은 상태 전체를 디버깅입니다. updates는 노드별 변경분입니다. 판단 기준은 실행 진행 로그입니다. messages는 LLM 토큰과 메타데이터입니다. 판단 기준은 채팅 UI 토큰 출력입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 52. updates 스트림은 어느 노드가 어떤 상태를 바꿨는지 보여줍니다

updates 스트림은 어느 노드가 어떤 상태를 바꿨는지 다룹니다. 복잡한 그래프에서는 최종값보다 노드별 변경분이 디버깅에 더 유용합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 stream은 invoke와 같은 입력을 받지만 iterator를 반환합니다. updates는 각 청크는 노드 이름과 변경된 필드를 담습니다. UI는 노드 이름을 단계 표시로 보여줄 수 있습니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 stream입니다. 노트북 기준 위치는 07_streaming.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 53. version='v2'는 스트림 이벤트를 type·ns·data로 통일합니다

version='v2'는 스트림 이벤트를 type·ns·data로 통일합니다. 서브그래프와 메시지 스트림이 섞여도 같은 구조로 분기합니다. 묶음별로 읽으면 StreamPart 구조에서는 chunk['type']. values, updates, messages, custom 같은 이벤트 종류입니다; chunk['ns']. 서브그래프 namespace가 있을 때 어느 그래프에서 온 이벤트인지 알려줍니다; chunk['data']. 실제 상태, 메시지, 커스텀 payload입니다를 봅니다 nostream 태그는 백그라운드 모델 토큰을 숨깁니다 여기서는 내부 분류기나 요약기 토큰이 사용자 채팅 UI에 섞이지 않게 합니다; metadata의 tags와 chunk_position으로 특정 모델의 마지막 토큰을 감지합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 54. 앱 스트림은 projection을 정해 사용자에게 필요한 상태만 보냅니다

앱 스트림은 projection을 정해 사용자에게 필요한 상태만 보냅니다. Event Streaming v3 관점에서는 그래프 내부 상태 전체가 아니라 화면 상태를 설계합니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 GRAPH 단계에서는 내부 상태를 다룹니다. PROJECT 단계에서는 화면 모델를 다룹니다. EVENT 단계에서는 프론트 전송를 다룹니다. RENDER 단계에서는 UI 반영를 다룹니다. 좋은 스트리밍은 많은 데이터를 보내는 것이 아니라 사용자가 이해할 사건만 보내는 것입니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

## 55. Interrupt와 Time Travel

이제 PART 09 Interrupt와 Time Travel로 넘어갑니다. 사람 승인, 리뷰·편집, 도구 interrupt, 입력 검증, replay와 fork를 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/08_interrupts_and_time_travel.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 56. interrupt 패턴은 승인·수정·입력·검증·도구 승인으로 나눠 볼 수 있습니다

interrupt 패턴은 승인, 수정, 입력, 검증, 도구 승인으로 나눠 볼 수 있습니다. 최신 LangGraph에서는 여기에 response_schema를 더해 재개 값의 형태를 명시할 수 있습니다. UI가 어떤 폼을 보여줘야 하는지도 이 스키마를 보고 결정할 수 있습니다.

## 57. Command(resume=...)은 중단된 그래프에 사람의 결정을 다시 넣습니다

Command resume 패턴은 그대로 유지됩니다. 여기에 response_schema를 넣으면 사람이 돌려줄 값의 구조를 코드로 표현할 수 있습니다. 예를 들어 승인 여부와 사유를 함께 받아야 한다면 TypedDict나 Pydantic 모델로 resume 값을 제한합니다.

## 58. Human-in-the-loop는 그래프 실행을 승인 가능한 지점에서 멈춥니다

Human-in-the-loop는 그래프 실행을 승인 가능한 지점에서 멈춥니다. interrupt는 예외가 아니라 체크포인트와 함께 쓰는 정상 제어 흐름입니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 interrupt는 사람에게 보여줄 값과 질문을 반환합니다. checkpoint는 중단 직전 상태가 저장되어 있어야 합니다. Command(resume)는 사람의 답을 같은 thread_id로 전달해 재개합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

## 59. 표준 interrupt 패턴은 resume 값의 모양으로 구분됩니다

표준 interrupt 패턴은 resume 값의 모양으로 구분됩니다. 승인·편집·도구 승인·입력 검증·다중 interrupt를 모두 같은 재개 원리로 다룹니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Approval은 True 또는 False입니다. 판단 기준은 메일 발송 전 승인입니다. Review & Edit는 수정된 문자열입니다. 판단 기준은 초안 문장 편집입니다. Tool Interrupt는 action dict입니다. 판단 기준은 도구 인자 일부 수정 후 승인입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 60. 타임 트래블은 replay와 fork를 구분해야 합니다

타임 트래블은 replay와 fork를 구분해야 합니다. replay는 과거 지점부터 재실행하고, fork는 과거 상태를 수정한 새 분기를 만듭니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 화면에서 Replay는 과거 checkpoint config로 invoke(None, config)를 호출합니다. Fork는 update_state로 값을 바꾸고 새 checkpoint를 만듭니다. as_node는 어느 노드가 상태를 쓴 것처럼 기록할지 지정합니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 invoke입니다. 노트북 기준 위치는 08_interrupts_and_time_travel.ipynb Replay vs Fork입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 61. interrupt는 response_schema와 함께 UI 계약이 됩니다

완료와 인터럽트는 반환 타입에서 먼저 구분합니다. 인터럽트가 있다면 value와 response_schema를 보고 UI를 구성하고, 같은 config와 Command resume으로 이어갑니다. 완료되었을 때만 최종 상태를 읽습니다.

## 62. 서브그래프

이제 PART 10 서브그래프로 넘어갑니다. 상태 스키마가 다른 서브그래프 래핑, 직접 노드 추가, 서브그래프 스트리밍과 checkpointer 모드를 다룹니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/09_subgraphs.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 63. 서브그래프 연결 방식은 상태 스키마가 같은지에 따라 달라집니다

서브그래프 연결 방식은 상태 스키마가 같은지에 따라 달라집니다. 큰 그래프를 나눌 때는 코드 재사용보다 상태 경계를 먼저 봐야 합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 직접 추가는 부모와 자식 상태 키가 맞습니다. 판단 기준은 공유 키의 reducer 충돌을 확인합니다. Wrapper node는 상태 스키마가 다릅니다. 판단 기준은 입력 변환과 출력 매핑을 명시합니다. LLM subgraph는 전문 흐름을 독립 그래프로 둡니다. 판단 기준은 프롬프트와 메시지 경계를 분리합니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 64. 서브그래프는 파일 분리보다 실행 경계 설계가 중요합니다

서브그래프는 파일 분리보다 실행 경계 설계가 중요합니다. 서브그래프를 만들면 독립성이 생기지만 상태 변환과 디버깅 복잡도도 늘어납니다. 묶음별로 읽으면 분리하면 좋은 경우에서는 팀이나 도메인이 다른 실행 흐름입니다; 하위 그래프만 따로 테스트하고 싶습니다; 반복 재사용되는 전문 워크플로입니다를 봅니다. 분리하지 않는 편이 나은 경우에서는 상태 키 몇 개만 추가하면 되는 단순 단계입니다; 부모와 자식이 거의 모든 상태를 공유합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 65. 서브그래프는 그래프 안에 들어가는 독립 그래프입니다

서브그래프는 그래프 안에 들어가는 독립 그래프입니다. 작은 그래프를 따로 컴파일해 부모 그래프 노드처럼 사용합니다. 핵심 레이어는 다음과 같습니다. Call inside node는 상태가 다를 때에 해당합니다. 부모 상태를 서브그래프 입력으로 변환하고 결과를 다시 부모 상태로 합칩니다. 코드에서는 subgraph.invoke(...) 같은 형태로 드러납니다. Add as node는 상태가 맞을 때에 해당합니다. 공유 키가 있는 그래프는 add_node(name, subgraph)로 직접 추가합니다. 코드에서는 builder.add_node('sub', subgraph) 같은 형태로 드러납니다. Namespace는 관측과 저장에 해당합니다. 서브그래프 스트림과 체크포인트는 namespace로 어느 내부 그래프인지 구분합니다. 코드에서는 subgraphs=True 같은 형태로 드러납니다. 이 구조를 잡아 두면 뒤에서 API가 늘어나도 같은 기준으로 분류합니다.

## 66. 상태 스키마가 다르면 래퍼 노드가 부모와 자식 상태를 변환합니다

상태 스키마가 다르면 래퍼 노드가 부모와 자식 상태를 변환합니다. 서브그래프 내부 구조는 숨기되, 부모 그래프 입출력 계약은 명확히 둡니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 subgraph는 분석 전용 그래프를 별도 컴파일합니다. wrapper은 부모 state를 서브그래프 입력으로 바꿉니다. merge는 서브그래프 결과만 부모 state에 되돌립니다. 코드는 다음 지점을 중심으로 읽습니다. 함수 run_analysis는 실제

작업 단위입니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. 노트북 기준 위치는 09_subgraphs.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 67. 서브그래프 checkpointer는 세 가지 모드로 생각합니다

서브그래프 checkpointer는 세 가지 모드로 생각합니다. 메모리와 interrupt 가능 여부가 모드마다 달라집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Per-Invocation은 checkpointer 미지정입니다. 판단 기준은 호출마다 fresh, 부모 checkpointer 상속입니다. Per-Thread는 checkpointer=True입니다. 판단 기준은 thread별 누적 메모리, 병렬 tool 호출은 피함입니다. Stateless는 checkpointer=False입니다. 판단 기준은 checkpoint가 없어 interrupt 불가입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

### 68. 여러 per-thread 서브그래프는 고유 노드 이름으로 namespace를 격리합니다

여러 per-thread 서브그래프는 고유 노드 이름으로 namespace를 격리합니다. agent 이름과 StateGraph 노드 이름을 맞추면 checkpoint prefix가 안정적으로 나뉩니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 name는 create_agent와 add_node에 같은 고유 이름을 씁니다. wrap는 각 서브에이전트를 작은 StateGraph로 감쌉니다. isolate는 fruit_agent와 veggie_agent의 state가 섞이지 않습니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 graph에서 MessagesState, StateGraph; agents에서 create_agent를 준비합니다. 함수 create_sub_agent는 실제 작업 단위입니다. 핵심 API는 create_agent, StateGraph, compile, add_node, add_edge입니다. 노트북 기준 위치는 09_subgraphs.ipynb Namespace입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 69. 프로덕션 운영

이제 PART 11 프로덕션 운영으로 넘어갑니다. langgraph.json, Studio, Agent Chat UI, GenericFakeChatModel, LangSmith tracing을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/10_production.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

### 70. langgraph.json은 그래프를 서버와 Studio가 찾을 수 있게 하는 배포 계약입니다

langgraph.json은 그래프를 서버와 Studio가 찾을 수 있게 하는 배포 계약입니다. 노트북 코드가 앱이 되려면 그래프 경로, 의존성, 환경 변수가 명시돼야 합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 화면에서 graphs는 그래프 이름과 Python 객체 경로를 연결합니다. dependencies는 서버가 설치할 패키지나 로컬 경로입니다. env는 실행에 필요한 환경 변수 파일입니다. 노트북 기준 위치는 10_production.ipynb langgraph.json입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 71. 프로덕션 테스트는 결정론 테스트와 LLM 테스트를 분리해야 합니다

프로덕션 테스트는 결정론 테스트와 LLM 테스트를 분리해야 합니다. 그래프 구조가 맞는지와 모델 답변이 좋은지는 서로 다른 검증입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 결정론 테스트는 상태 전이, 라우팅, reducer입니다. 판단 기준은 일반 unit test입니다. LLM 에이전트 테스트는 도구 호출 순서와 최종 메시

지입니다. 판단 기준은 GenericFakeChatModel입니다. 회귀 테스트는 이전 질문 세트의 품질 유지입니다. 판단 기준은 dataset/eval입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

### 72. LangGraph 앱 구조는 그래프 객체를 서버와 Studio가 찾을 수 있게 만듭니다

LangGraph 앱 구조는 그래프 객체를 서버와 Studio가 찾도록 만듭니다. 코드에서 graph를 export하고 langgraph.json이 그 위치를 가리킵니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 langgraph.json은 dependencies, graphs, env 선언입니다. 판단 기준은 서버가 graph를 import 가능함입니다. LangGraph Studio는 로컬 그래프 시각 디버깅입니다. 판단 기준은 상태와 노드 실행을 눈으로 확인입니다. Agent Chat UI는 채팅형 테스트 표면입니다. 판단 기준은 streaming과 thread 상태를 빠르게 점검입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

### 73. langgraph.json은 그래프 엔트리포인트를 선언하는 배포 계약입니다

langgraph.json은 그래프 엔트리포인트를 선언하는 배포 계약입니다. 로컬 개발 서버와 Studio는 이 파일을 기준으로 그래프를 로드합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 dependencies는 앱 실행에 필요한 패키지 또는 현재 프로젝트를 지정합니다. graphs는 그래프 이름과 Python import 경로를 매핑합니다. env는 환경변수 파일을 지정합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 json를 준비합니다. 노트북 기준 위치는 10_production.ipynb langgraph.json입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 74. GenericFakeChatModel은 LLM 테스트를 결정론적으로 만듭니다

GenericFakeChatModel은 LLM 테스트를 결정론적으로 만듭니다. 모델 답을 고정하면 그래프 라우팅과 상태 업데이트를 자동 테스트합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 fake model은 예상 메시지 iterator를 모델처럼 씁니다. assert는 결과 메시지와 state를 검증합니다. 비용 없음은 실제 API 호출 없이 CI에서 반복 실행합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 language_models에서 GenericFakeChatModel; messages에서 AIMessage, HumanMessage를 준비합니다. 함수 chatbot는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 노트북 기준 위치는 10_production.ipynb 테스트입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 75. 관측성은 trace를 남기는 것보다 민감정보를 남기지 않는 설계가 먼저입니다

관측성은 trace를 남기는 것보다 민감정보를 남기지 않는 설계가 먼저입니다. LangSmith tracing은 강력하지만 anonymizer와 metadata 정책을 함께 정해야 합니다. 묶음별로 읽으면 Selective tracing에서는 tracing_context로 특정 호출만 켜고 끌 수 있습니다; project, tag, metadata를 동적으로 부여해 실험과 운영을 분리합니다를 봅니다. Data privacy에서는 LangChainTracer와 Client anonymizer로 로그 직전 마스킹을 적용합니다; 사용자 입력 전체를 남길지, 추출 필드만 남길지 팀 정책을 먼저 정합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

---

### 76. 로컬 서버

이제 PART 12 로컬 서버로 넘어갑니다. LangGraph CLI, langgraph dev, Studio, Python SDK, REST API 호출 흐름을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/11_local_server.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면

코드의 위치가 분명해집니다.

---

### 77. 로컬 서버는 같은 그래프를 Studio, SDK, REST에서 호출하게 합니다

로컬 서버는 같은 그래프를 Studio, SDK, REST에서 호출하게 합니다. 강의용 노트북을 서비스 표면으로 바꾸는 중간 단계입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 LangGraph CLI는 개발 서버 실행입니다. 판단 기준은 langgraph dev입니다. Studio는 그래프 시각 디버깅입니다. 판단 기준은 상태와 노드 이동 확인입니다. Python SDK는 비동기 클라이언트 호출입니다. 판단 기준은 threads와 runs입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

### 78. SDK 호출은 thread 생성과 run stream을 분리해서 이해해야 합니다

SDK 호출은 thread 생성과 run stream을 분리해서 이해해야 합니다. 서버 모드에서는 그래프 실행이 로컬 함수 호출이 아니라 원격 run으로 관리됩니다. 묶음별로 읽으면 기본 순서에서는 client를 생성합니다; thread를 만듭니다; assistant 또는 graph id로 run을 시작합니다를 봅니다. 운영 관점에서는 thread는 대화와 실행 상태의 단위입니다; run id는 개별 실행 추적 단위입니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

---

### 79. 로컬 서버는 노트북 그래프를 서비스 호출 표면으로 바꿉니다

로컬 서버는 노트북 그래프를 서비스 호출 표면으로 바꿉니다. 개발 중에는 in-memory 서버로 빠르게 돌리고, 배포 전에는 동일 API로 검증합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 CLI 설치는 pip install 'langgraph-cli[inmem]'입니다. 판단 기준은 langgraph 명령 사용 가능입니다. 프로젝트 생성은 agent.py + langgraph.json입니다. 판단 기준은 그래프 import 경로 확인입니다. 개발 서버는 langgraph dev입니다. 판단 기준은 localhost:2024 실행입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

### 80. Python SDK는 서버 실행 중인 그래프를 비동기 스트림으로 호출합니다

Python SDK는 서버 실행 중인 그래프를 비동기 스트림으로 호출합니다. 노트북의 graph.invoke 감각을 서버 API로 옮기는 단계입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 get_client는 로컬 서버 URL을 지정합니다. runs.stream은 그래프 이름과 input을 넘기고 이벤트를 순회합니다. event/data는 프론트엔드와 백엔드는 같은 event payload를 기준으로 연결됩니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 langgraph_sdk에서 get_client; asyncio를 준비합니다. 함수 main는 실제 작업 단위입니다. 핵심 API는 stream입니다. 노트북 기준 위치는 11_local_server.ipynb SDK입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

### 81. 동기 SDK와 REST는 같은 실행을 다른 클라이언트 표면으로 부릅니다

동기 SDK와 REST는 같은 실행을 다른 클라이언트 표면으로 부릅니다. 서버를 붙인 뒤에는 SDK, REST, UI가 같은 graph_id와 thread를 공유합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 sync client는 스크립트와 테스트에서 간단히 사용합니다. stream_mode는 messages-tuple 같은 UI 친화 모드를 지정합니다. REST는 언어와 관계없이 배포된 그래프를 호출합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 langgraph_sdk에서 get_sync_client를 준비합니다. 핵심 API는 stream입니다. 노트북 기준 위치는 11_local_server.ipynb Sync client입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 82. 배포 준비는 서버가 켜지는지보다 호출 계약이 안정적인지를 봅니다

배포 준비는 서버가 켜지는지보다 호출 계약이 안정적인지를 봅니다. 입력 스키마, graph 이름, env, thread 정책이 흔들리면 UI와 운영 자동화가 함께 깨집니다. 묶음별로 읽으면 배포 전 체크에서는 langgraph.json의 graph 경로가 실제 객체를 import하는지 확인합니다; 필수 환경변수와 선택 환경변수를 분리합니다; SDK와 REST 모두 같은 샘플 입력으로 성공하는지 확인합니다를 봅니다. 운영 연결에서는 Studio는 디버깅용, SDK/REST는 앱 연동용으로 역할을 나눕니다; thread_id와 user context 정책을 문서화해야 상태 격리가 유지됩니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 83. 내구성 실행

이제 PART 13 내구성 실행으로 넘어갑니다. @task, 체크포인터, fault tolerance, graceful shutdown, DeltaChannel을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/12_durable_execution.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 84. 내구성 실행은 실패 복구를 위해 side effect와 checkpoint를 분리합니다

내구성 실행은 실패 복구를 위해 side effect와 checkpoint를 분리합니다. 긴 작업은 처음부터 재실행해도 안전한 코드 구조가 필요합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Idempotency는 같은 작업을 다시 해도 결과가 망가지지 않습니다. 판단 기준은 외부 쓰기 작업은 고유 id로 보호입니다. Checkpoint는 어디까지 끝났는지 저장합니다. 판단 기준은 checkpointer와 thread_id 사용입니다. Task boundary는 재시도 가능한 단위를 분리합니다. 판단 기준은 @task로 외부 작업 분리입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 85. @task로 외부 작업을 감싸면 재시도와 재개 경계가 선명해집니다

@task로 외부 작업을 감싸면 재시도와 재개 경계가 선명해집니다. entrypoint 안에 모든 작업을 몰아넣으면 실패 후 어디부터 다시 시작할지 불분명합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 @task는 실패하거나 오래 걸리는 외부 작업 단위입니다. entrypoint는 작업 순서를 조립하는 진입점입니다. checkpointer은 완료된 task 결과를 저장합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 call_external_api, durable_flow는 실제 작업 단위입니다. 노트북 기준 위치는 12_durable_execution.ipynb durable task입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 86. Graph API와 Functional API는 재개 시작점이 다릅니다

Graph API와 Functional API는 재개 시작점이 다릅니다. 어디서부터 다시 실행되는지 알아야 중복 결제나 중복 파일 쓰기를 피합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 StateGraph는 중단된 노드의 시작입니다. 판단 기준은 노드 내부 side effect는 idempotent해야 함입니다. Functional API는 entrypoint에서 시작입니다. 판단 기준은 @task 결과는 checkpoint에서 복원입니다. @task는 작업 결과 저장입니다. 판단 기준은 비결정적 IO와 LLM 호출을 task 안에 둠입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 87. Functional API에서는 LLM 호출과 외부 IO를 @task로 감싸야 재개가 안정적입니다

Functional API에서는 LLM 호출과 외부 IO를 @task로 감싸야 재개가 안정적입니다. entrypoint가 다시 시작되어도 이미 끝난 task 결과는 저장값에서 복원됩니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 generate_draft는 비싼 작업 또는 LLM 호출을 task로 둡니다. checkpointer은 entrypoint에 연결합니다. thread_id는 장애 후 같은 실행으로 재개합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 generate_draft, review_draft, write_document는 실제 작업 단위입니다. 노트북 기준 위치는 12_durable_execution.ipynb @task입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 88. LangGraph 1.2 fault tolerance는 timeout, retry, error handler를 더 명시적으로 다룹니다

LangGraph 1.2 fault tolerance는 timeout, retry, error handler를 더 명시적으로 다룹니다. 장기 작업은 실패를 숨기는 것이 아니라 보정 가능한 상태로 남기는 설계가 필요합니다. 묶음별로 읽으면 TimeoutPolicy에서는 idle_timeout으로 노드가 멈춰 있는 시간을 제한합니다; heartbeat를 쓰면 장기 작업 중에도 살아 있음을 런타임에 알릴 수 있습니다를 봅니다. Graceful shutdown에서는 RunControl로 drain을 요청하고 GraphDrained를 받아 checkpoint 기반 재개를 준비합니다; runtime.drain_requested를 보고 노드 내부에서 안전하게 중단합니다를 봅니다. DeltaChannel에서는 append-heavy 메시지나 로그에서 checkpoint 저장량을 줄이는 beta 채널입니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 89. 장애 복구 시나리오는 체크포인트 확인에서 시작합니다

장애 복구 시나리오는 체크포인트 확인에서 시작합니다. 실패 후 처음 할 일은 재시작이 아니라 어떤 노드까지 완료됐는지 확인하는 것입니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 FAIL 단계에서는 노드 실패를 다룹니다. INSPECT 단계에서는 get_state를 다룹니다. FIX 단계에서는 update_state를 다룹니다. RESUME 단계에서는 invoke(None)를 다룹니다. 내구성 실행은 자동 재시도만이 아니라 상태를 보고 고친 뒤 재개하는 운영 절차입니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

## 90. API 선택과 Pregel

이제 PART 14 API 선택과 Pregel로 넘어갑니다. Graph API, Functional API, 두 API 결합, Pregel 채널과 슈퍼스텝 실행 모델을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/13_api_guide_and_pregel.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

## 91. Graph API와 Functional API 선택은 팀이 읽을 실행 표현을 기준으로 합니다

Graph API와 Functional API 선택은 표현 방식의 문제입니다. 운영에서는 여기에 tracing 기준도 붙습니다. StateGraph.add_node의 trace_policy로 노드별 추적 payload를 조정할 수 있고, 민감하거나 큰 입력은 줄여서 남길 수 있습니다.

## 92. Pregel은 노드가 superstep 단위로 메시지를 주고받는 내부 실행 모델입니다

Pregel은 노드가 superstep 단위로 메시지를 주고받는 내부 실행 모델입니다. LangGraph를 깊게 디버깅할 때는 그래프가 한 번에 전부 실행되는 것이 아니라 단계별로 동기화된다고 이해해야 합니다. 묶음별로 읽으면 핵심 관점에서는 각 superstep에서 활성 노드가 실행됩니다; 노드는 채널에 값을 쓰고 다음 superstep에서 읽힙니다; 더 이상 활성 노드가 없으면 실행이 끝납니다를 봅니다. 학습 효과에서는 reducer와 channel의 의미가 선명해집니다; 병렬 fan-out/fan-in의 실행 순서를 설명합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

## 93. Graph API와 Functional API 선택은 팀의 디버깅 표면을 고르는 문제입니다

Graph API와 Functional API 선택은 팀의 디버깅 표면을 고르는 문제입니다. 기능 차이보다 복잡성이 어디에 보이는지가 선택 기준입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 복잡한 분기는 노드와 엣지로 드러남입니다. 판단 기준은 함수 내부 if/while로 숨어듦입니다. 간단한 파이프라인은 장황해질 위험입니다. 판단 기준은 읽기 쉽고 짧음입니다. 시각화는 Studio에서 구조가 잘 보임입니다. 판단 기준은 task 단위 흐름 중심입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 94. 두 API는 한 앱 안에서 역할을 나눠 함께 쓸 수 있습니다

두 API는 한 앱 안에서 역할을 나눠 함께 쓸 수 있습니다. 복잡한 조정은 Graph API, 단순 데이터 처리 worker는 Functional API로 분리합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 Graph API는 상위 coordinator와 라우팅을 담당합니다. Functional API는 반복 가능한 worker 파이프라인을 담당합니다. 계약은 서로 넘기는 입력과 출력 스키마만 고정합니다. 코드는 다음 지점을 중심으로 읽습니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 summarize, summarize_worker, worker_node는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, invoke, add_node입니다. 노트북 기준 위치는 13_api_guide_and_pregel.ipynb API 결합입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

## 95. Pregel은 노드가 채널을 읽고 쓰는 슈퍼스텝 실행 모델입니다

Pregel은 노드가 채널을 읽고 쓰는 슈퍼스텝 실행 모델입니다. StateGraph를 쓰면 직접 Pregel을 다루지 않아도 내부 모델을 이해합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 NodeBuilder은 채널 구독과 쓰기 선언입니다. 판단 기준은 subscribe_only('a').write_to('b')입니다. Channel은 값 전달과 누적 방식입니다. 판단 기준은 EphemeralValue, LastValue, Topic입니다. Superstep는 같은 단계의 노드가 병렬 실행입니다. 판단 기준은 업데이트는 다음 step에서 관찰입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 96. 개념 근거와 실행 모델

이제 REFERENCE 개념 근거와 실행 모델으로 넘어갑니다. LangGraph 개념 설명은 Pregel 논문, LangGraph 공식 문서, ReAct 논문을 기준으로 보강합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 여기서는 개념 설명의 근거를 정리합니다. 실무에서 프레임워크를 설명할 때는 개인적인 비유보다 공식 문서와 논문에 기대는 편이 안전합니다.

## 97. LangGraph의 superstep 관점은 Pregel의 반복 실행 모델로 설명할 수 있습니다

LangGraph의 superstep 관점은 Pregel의 반복 실행 모델로 설명합니다. Pregel은 그래프 계산을 superstep의 반복으로 설명합니다. LangGraph의 Pregel API와 노드 실행 이해에 유용한 배경입니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Superstep는 활성 노드가 실행되고 값을 채널에 씁니다. Message/Channel은 이번 단계의 출력은 다음 단계의 입력이 됩니다. Halt는 더 실행할 노드가 없으면 그래프가 종료됩니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

## 98. Persistence와 time travel은 checkpoint를 기준으로 실행을 조회·수정·재개합니다

Persistence와 time travel은 checkpoint를 기준으로 실행을 조회·수정·재개합니다. LangGraph 공식 문서는 checkpointer를 thread-scoped short-term memory, Store를 cross-thread long-term memory로 구분합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Checkpoint는 각 실행 단계의 상태 스냅샷입니다. Interrupt는 사람 입력이 필요한 지점에서 실행을 멈춥니다. Replay는 이전 상태를 조회하거나 수정해 다른 경로를 실험합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

## 99. LangGraph 개념 보강 출처는 논문과 공식 문서를 기준으로 삼습니다

LangGraph 개념 보강 출처는 논문과 공식 문서를 기준으로 삼습니다. 덱의 개념 설명은 노트북 코드와 아래 신뢰 출처를 함께 반영합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 ReAct loop는 Yao et al., ReAct, arXiv:2210.03629입니다. 판단 기준은 reasoning-action-observation 루프 설명입니다. RAG는 Lewis et al., RAG, NeurIPS 2020입니다. 판단 기준은 검색 증강 생성과 provenance 설명입니다. Pregel은 Malewicz et al., SIGMOD 2010입니다. 판단 기준은 superstep, message passing, halt 설명입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

## 100. 01_introduction.ipynb

이제 CODE 01_introduction.ipynb로 넘어갑니다. 01_introduction.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/01_introduction.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

## 101. 01_introduction 코드 01

이제 01_introduction 코드 01 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 importlib를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 102. 01_introduction 코드 02

이제 01_introduction 코드 02 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 103. 01_introduction 코드 03

이제 01_introduction 코드 03 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 104. 01_introduction 코드 04

이제 01_introduction 코드 04 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; func에서 entrypoint, task를 준비합니다. 핵심 API는 StateGraph, ChatOpenAI, InMemorySaver, interrupt, Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 105. 01_introduction 코드 05

이제 01_introduction 코드 05 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; typing에서 TypedDict를 준비합니다. 클래스 SimpleState는 데이터 구조나 정책 단위를 정의합니다. 함수 greet는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 106. 01_introduction 코드 06

이제 01_introduction 코드 06 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 process, simple_workflow는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 107. 02_graph_api.ipynb

이제 CODE 02_graph_api.ipynb로 넘어갑니다. 02_graph_api.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/02_graph_api.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

### 108. 02_graph_api 코드 01

이제 02_graph_api 코드 01 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 109. 02_graph_api 코드 02

이제 02_graph_api 코드 02 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 110. 02_graph_api 코드 03 (1/2)

이제 02_graph_api 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; typing에서 TypedDict를 준비합니다. 클래스 State는 데이터 구조나 정책 단위를 정의합니다. 함수 count_words는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 111. 02_graph_api 코드 03 (2/2)

이제 02_graph_api 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 112. 02_graph_api 코드 04 (1/3)

이제 02_graph_api 코드 04 (1/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Annotated, TypedDict; operator를 준비합니다. 클래스 AccState는 데이터 구조나 정책 단위를 정의합니다. 함수 step_a, step_b는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 113. 02_graph_api 코드 04 (2/3)

이제 02_graph_api 코드 04 (2/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 114. 02_graph_api 코드 04 (3/3)

이제 02_graph_api 코드 04 (3/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 115. 02_graph_api 코드 05 (1/2)

이제 02_graph_api 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Literal를 준비합니다. 클래스 RouterState는 데이터 구조나 정책 단위를 정의합니다. 함수 classify, handle_weather, handle_math, handle_general는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 116. 02_graph_api 코드 05 (2/2)

이제 02_graph_api 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 route는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge, add_conditional_edges입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 117. 02_graph_api 코드 06

이제 02_graph_api 코드 06 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 MessagesState; messages에서 HumanMessage, SystemMessage를 준비합니다. 함수 chatbot는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 118. 02_graph_api 코드 07 (1/3)

이제 02_graph_api 코드 07 (1/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 TypedDict를 준비합니다. 클래스 InputSchema, OutputSchema, InternalState는 데이터 구조나 정책 단위를 정의합니다. 함수 process는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 119. 02_graph_api 코드 07 (2/3)

이제 02_graph_api 코드 07 (2/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 answer는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 120. 02_graph_api 코드 07 (3/3)

이제 02_graph_api 코드 07 (3/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 121. 02_graph_api 코드 08 (1/2)

이제 02_graph_api 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dataclasses에서 dataclass; runtime에서 Runtime를 준비합니다. 클래스 ContextSchema, CtxState는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 ctx_node는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 122. 02_graph_api 코드 08 (2/2)

이제 02_graph_api 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 123. 02_graph_api 코드 09 (1/3)

이제 02_graph_api 코드 09 (1/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 RetryPolicy, CachePolicy; memory에서 InMemoryCache를 준비합니다. 클래스 ReliableState는 데이터 구조나 정책 단위를 정의합니다. 함수 flaky, heavy는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 124. 02_graph_api 코드 09 (2/3)

이제 02_graph_api 코드 09 (2/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 125. 02_graph_api 코드 09 (3/3)

이제 02_graph_api 코드 09 (3/3) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 126. 02_graph_api 코드 10 (1/2)

이제 02_graph_api 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Send, Command를 준비합니다. 클래스 FanOutState는 데이터 구조나 정책 단위를 정의합니다. 함수 dispatch, worker, reduce는 실제 작업 단위입니다. 핵심 API는 StateGraph, Command, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 127. 02_graph_api 코드 10 (2/2)

이제 02_graph_api 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 02_graph_api.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_node, add_edge, add_conditional_edges입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 128. 03_functional_api.ipynb

이제 CODE 03_functional_api.ipynb로 넘어갑니다. 03_functional_api.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/03_functional_api.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

## 129. 03_functional_api 코드 01

이제 03_functional_api 코드 01 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 130. 03_functional_api 코드 02

이제 03_functional_api 코드 02 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 131. 03_functional_api 코드 03 (1/2)

이제 03_functional_api 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 fetch_data, process_data, data_pipeline는 실제 작업 단위입니다. 핵심 API는 InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 132. 03_functional_api 코드 03 (2/2)

이제 03_functional_api 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 133. 03_functional_api 코드 04 (1/3)

이제 03_functional_api 코드 04 (1/3) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, task, task가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 analyze_sentiment, extract_keywords, summarize, parallel_analysis는 실제 작업 단위입니다. 핵심 API는 InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 134. 03_functional_api 코드 04 (2/3)

이제 03_functional_api 코드 04 (2/3) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 135. 03_functional_api 코드 04 (3/3)

이제 03_functional_api 코드 04 (3/3) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 136. 03_functional_api 코드 05

이제 03_functional_api 코드 05 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 데코레이터 entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 counter는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 137. 03_functional_api 코드 06

이제 03_functional_api 코드 06 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 데코레이터 entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 smart_counter는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 138. 03_functional_api 코드 07 (1/2)

이제 03_functional_api 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 time를 준비합니다. 데코레이터 task, task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_timestamp, call_llm, deterministic_workflow는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 139. 03_functional_api 코드 07 (2/2)

이제 03_functional_api 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 140. 03_functional_api 코드 08 (1/3)

이제 03_functional_api 코드 08 (1/3) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool; messages에서 HumanMessage, SystemMessage, ToolMessage를 준비합니다. 데코레이터 tool, tool, task가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add, multiply, call_model는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 141. 03_functional_api 코드 08 (2/3)

이제 03_functional_api 코드 08 (2/3) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 call_tool, agent는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 142. 03_functional_api 코드 08 (3/3)

이제 03_functional_api 코드 08 (3/3) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 143. 03_functional_api 코드 09

이제 03_functional_api 코드 09 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 RetryPolicy를 준비합니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 fetch_with_retry, reliable_pipeline는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 144. 03_functional_api 코드 10 (1/2)

이제 03_functional_api 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 memory에서 InMemoryCache; types에서 CachePolicy를 준비합니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 expensive_compute, cached_pipeline는 실제 작업 단위입니다. 핵심 API는 InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 145. 03_functional_api 코드 10 (2/2)

이제 03_functional_api 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 146. 03_functional_api 코드 11 (1/2)

이제 03_functional_api 코드 11 (1/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command, interrupt를 준비합니다. 데코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 review, approval_flow는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, interrupt, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 147. 03_functional_api 코드 11 (2/2)

이제 03_functional_api 코드 11 (2/2) 슬라이드입니다. 노트북 위치는 03_functional_api.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, interrupt, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 148. 04_workflows.ipynb

이제 CODE 04_workflows.ipynb로 넘어갑니다. 04_workflows.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/04_workflows.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

## 149. 04_workflows 코드 01

이제 04_workflows 코드 01 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 150. 04_workflows 코드 02

이제 04_workflows 코드 02 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 151. 04_workflows 코드 03 (1/2)

이제 04_workflows 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; typing에서 TypedDict를 준비합니다. 클래스 ChainState는 데이터 구조나 정책 단위를 정의합니다. 함수 generate_draft, improve_draft는 실제 작업 단위입니다. 핵심 API는 StateGraph, invoke, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 152. 04_workflows 코드 03 (2/2)

이제 04_workflows 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 153. 04_workflows 코드 04 (1/3)

이제 04_workflows 코드 04 (1/3) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Annotated, TypedDict; operator를 준비합니다. 클래스 ParallelState는 데이터 구조나 정책 단위를 정의합니다. 함수 analyze_tone, analyze_complexity는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 154. 04_workflows 코드 04 (2/3)

이제 04_workflows 코드 04 (2/3) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 synthesize는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 155. 04_workflows 코드 04 (3/3)

이제 04_workflows 코드 04 (3/3) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 156. 04_workflows 코드 05 (1/4)

이제 04_workflows 코드 05 (1/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 긴 코드 셀을 4장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 pydantic에서 BaseModel, Field; typing에서 Literal, TypedDict를 준비합니다. 클래스 Classification, RouteState는 데이터 구조나 정책 단위를 정의합니다. 함수 classify는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 157. 04_workflows 코드 05 (2/4)

이제 04_workflows 코드 05 (2/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 handle_technical, handle_business, handle_casual는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 158. 04_workflows 코드 05 (3/4)

이제 04_workflows 코드 05 (3/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 route는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, add_node, add_edge, add_conditional_edges입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 159. 04_workflows 코드 05 (4/4)

이제 04_workflows 코드 05 (4/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 160. 04_workflows 코드 06 (1/4)

이제 04_workflows 코드 06 (1/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 긴 코드 셀을 4장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Send; typing에서 Annotated, TypedDict를 준비합니다. 클래스 OrchestratorState, WorkerState는 데이터 구조나 정책 단위를 정의합니다. 함수 plan_sections는 실제 작업 단위입니다. 핵심 API는 invoke, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 161. 04_workflows 코드 06 (2/4)

이제 04_workflows 코드 06 (2/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 assign_workers, worker는 실제 작업 단위입니다. 핵심 API는 invoke, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 162. 04_workflows 코드 06 (3/4)

이제 04_workflows 코드 06 (3/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, add_node, add_edge, add_conditional_edges입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 163. 04_workflows 코드 06 (4/4)

이제 04_workflows 코드 06 (4/4) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 164. 04_workflows 코드 07 (1/2)

이제 04_workflows 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 클래스 EvalState는 데이터 구조나 정책 단위를 정의합니다. 함수 generate, evaluate, should_retry는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 165. 04_workflows 코드 07 (2/2)

이제 04_workflows 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 04_workflows.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge, add_conditional_edges입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 166. 05_agents.ipynb

이제 CODE 05_agents.ipynb로 넘어갑니다. 05_agents.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/05_agents.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

### 167. 05_agents 코드 01

이제 05_agents 코드 01 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 168. 05_agents 코드 02

이제 05_agents 코드 02 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 169. 05_agents 코드 03 (1/2)

이제 05_agents 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool; messages에서 HumanMessage, SystemMessage, ToolMessage를 준비합니다. 데코레이터 tool, tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add, multiply, divide는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 170. 05_agents 코드 03 (2/2)

이제 05_agents 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 171. 05_agents 코드 04 (1/2)

이제 05_agents 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; typing에서 TypedDict, Annotated, Literal를 준비합니다. 클래스 AgentState는 데이터 구조나 정책 단위를 정의합니다. 함수 llm_node, tool_node는 실제 작업 단위입니다. 핵심 API는 StateGraph, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 172. 05_agents 코드 04 (2/2)

이제 05_agents 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 should_continue는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge, add_conditional_edges입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 173. 05_agents 코드 05

이제 05_agents 코드 05 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 174. 05_agents 코드 06 (1/2)

이제 05_agents 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 call_model, execute_tool, functional_agent는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 175. 05_agents 코드 06 (2/2)

이제 05_agents 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 176. 05_agents 코드 07

이제 05_agents 코드 07 슬라이드입니다. 노트북 위치는 05_agents.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 memory에서 InMemorySaver를 준비합니다. 핵심 API는 InMemorySaver, compile, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 177. 06_persistence_and_memory.ipynb

이제 CODE 06_persistence_and_memory.ipynb로 넘어갑니다. 06_persistence_and_memory.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/06_persistence_and_memory.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

### 178. 06_persistence_and_memory 코드 01

이제 06_persistence_and_memory 코드 01 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 179. 06_persistence_and_memory 코드 02

이제 06_persistence_and_memory 코드 02 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 180. 06_persistence_and_memory 코드 03 (1/2)

이제 06_persistence_and_memory 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; memory에서 InMemorySaver를 준비합니다. 함수 chatbot는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 181. 06_persistence_and_memory 코드 03 (2/2)

이제 06_persistence_and_memory 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 182. 06_persistence_and_memory 코드 04

이제 06_persistence_and_memory 코드 04 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 183. 06_persistence_and_memory 코드 05

이제 06_persistence_and_memory 코드 05 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 설정값 configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 184. 06_persistence_and_memory 코드 06

이제 06_persistence_and_memory 코드 06 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 messages에서 AIMessage를 준비합니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 185. 06_persistence_and_memory 코드 07

이제 06_persistence_and_memory 코드 07 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 186. 06_persistence_and_memory 코드 08

이제 06_persistence_and_memory 코드 08 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 memory에서 InMemoryStore를 준비합니다. 핵심 API는 InMemoryStore입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 187. 06_persistence_and_memory 코드 09 (1/4)

이제 06_persistence_and_memory 코드 09 (1/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 4장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 dataclasses에서 dataclass; graph에서 StateGraph, START, END를 준비합니다. 클래스 Context는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 chatbot_with_memory는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 188. 06_persistence_and_memory 코드 09 (2/4)

이제 06_persistence_and_memory 코드 09 (2/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, invoke입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 189. 06_persistence_and_memory 코드 09 (3/4)

이제 06_persistence_and_memory 코드 09 (3/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable, context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 190. 06_persistence_and_memory 코드 09 (4/4)

이제 06_persistence_and_memory 코드 09 (4/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 191. 06_persistence_and_memory 코드 10 (1/2)

이제 06_persistence_and_memory 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; memory에서

InMemorySaver를 준비합니다. 함수 chatbot_with_trim는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 192. 06_persistence_and_memory 코드 10 (2/2)

이제 06_persistence_and_memory 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 193. 06_persistence_and_memory 코드 11 (1/2)

이제 06_persistence_and_memory 코드 11 (1/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; memory에서 InMemorySaver를 준비합니다. 함수 chatbot_node, delete_old_messages는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, invoke, add_node입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 194. 06_persistence_and_memory 코드 11 (2/2)

이제 06_persistence_and_memory 코드 11 (2/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 195. 06_persistence_and_memory 코드 12 (1/4)

이제 06_persistence_and_memory 코드 12 (1/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 4장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Annotated; typing_extensions에서 TypedDict를 준비합니다. 클래스 PipelineState는 데이터 구조나 정책 단위를 정의합니다. 함수 step_1는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 196. 06_persistence_and_memory 코드 12 (2/4)

이제 06_persistence_and_memory 코드 12 (2/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 step_2, step_3는 실제 작업 단위입니다. 핵심 API는 StateGraph입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 197. 06_persistence_and_memory 코드 12 (3/4)

이제 06_persistence_and_memory 코드 12 (3/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 198. 06_persistence_and_memory 코드 12 (4/4)

이제 06_persistence_and_memory 코드 12 (4/4) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 199. 06_persistence_and_memory 코드 13 (1/3)

이제 06_persistence_and_memory 코드 13 (1/3) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 postgres에서 PostgresSaver; aio에서 AsyncPostgresSaver를 준비합니다. 핵심 API는 compile입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 200. 06_persistence_and_memory 코드 13 (2/3)

이제 06_persistence_and_memory 코드 13 (2/3) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저

import는 mongodb에서 MongoDBSaver; redis에서 RedisSaver를 준비합니다. 핵심 API는 compile입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 201. 06_persistence_and_memory 코드 13 (3/3)

이제 06_persistence_and_memory 코드 13 (3/3) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 compile입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 202. 06_persistence_and_memory 코드 14 (1/2)

이제 06_persistence_and_memory 코드 14 (1/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 postgres에서 PostgresStore; redis에서 RedisStore를 준비합니다. 핵심 API는 compile입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 203. 06_persistence_and_memory 코드 14 (2/2)

이제 06_persistence_and_memory 코드 14 (2/2) 슬라이드입니다. 노트북 위치는 06_persistence_and_memory.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 embeddings에서 init_embeddings; memory에서 InMemoryStore를 준비합니다. 핵심 API는 InMemoryStore, compile입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 204. 07_streaming.ipynb

이제 CODE 07_streaming.ipynb로 넘어갑니다. 07_streaming.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/07_streaming.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

## 205. 07_streaming 코드 01 (1/3)

이제 07_streaming 코드 01 (1/3) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서

load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 search는 실제 작업 단위입니다. 핵심 API는 StateGraph, ChatOpenAI, InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 206. 07_streaming 코드 01 (2/3)

이제 07_streaming 코드 01 (2/3) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 클래스 AgentState는 데이터 구조나 정책 단위를 정의합니다. 함수 llm_node, tool_node, should_continue는 실제 작업 단위입니다. 핵심 API는 StateGraph, invoke, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 207. 07_streaming 코드 01 (3/3)

이제 07_streaming 코드 01 (3/3) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemorySaver, compile, add_edge, add_conditional_edges입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 208. 07_streaming 코드 02

이제 07_streaming 코드 02 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 209. 07_streaming 코드 03

이제 07_streaming 코드 03 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

## 210. 07_streaming 코드 04

이제 07_streaming 코드 04 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 211. 07_streaming 코드 05

이제 07_streaming 코드 05 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 212. 07_streaming 코드 06

이제 07_streaming 코드 06 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 213. 07_streaming 코드 07 (1/3)

이제 07_streaming 코드 07 (1/3) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 config에서 get_stream_writer를 준비합니다. 클래스 CustomState는 데이터 구조나 정책 단위를 정의합니다. 함수 research_node는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 214. 07_streaming 코드 07 (2/3)

이제 07_streaming 코드 07 (2/3) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, stream, add_node, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 215. 07_streaming 코드 07 (3/3)

이제 07_streaming 코드 07 (3/3) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 216. 07_streaming 코드 08 (1/2)

이제 07_streaming 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 217. 07_streaming 코드 08 (2/2)

이제 07_streaming 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 218. 07_streaming 코드 09

이제 07_streaming 코드 09 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 219. 07_streaming 코드 10

이제 07_streaming 코드 10 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 220. 07_streaming 코드 11 (1/2)

이제 07_streaming 코드 11 (1/2) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 metadata에서 version를 준비합니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 221. 07_streaming 코드 11 (2/2)

이제 07_streaming 코드 11 (2/2) 슬라이드입니다. 노트북 위치는 07_streaming.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 stream, interrupt, Command입니다. 설정값 thread_id는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 222. 08_interrupts_and_time_travel.ipynb

이제 CODE 08_interrupts_and_time_travel.ipynb로 넘어갑니다. 08_interrupts_and_time_travel.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/08_interrupts_and_time_travel.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

## 223. 08_interrupts_and_time_travel 코드 01

이제 08_interrupts_and_time_travel 코드 01 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 224. 08_interrupts_and_time_travel 코드 02

이제 08_interrupts_and_time_travel 코드 02 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 225. 08_interrupts_and_time_travel 코드 03 (1/4)

이제 08_interrupts_and_time_travel 코드 03 (1/4) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 4장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; types에서 interrupt,

Command를 준비합니다. 클래스 ReviewState는 데이터 구조나 정책 단위를 정의합니다. 함수 draft_document, human_review는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, interrupt, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 226. 08_interrupts_and_time_travel 코드 03 (2/4)

이제 08_interrupts_and_time_travel 코드 03 (2/4) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 finalize는 실제 작업 단위입니다. 핵심 API는 StateGraph입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 227. 08_interrupts_and_time_travel 코드 03 (3/4)

이제 08_interrupts_and_time_travel 코드 03 (3/4) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 228. 08_interrupts_and_time_travel 코드 03 (4/4)

이제 08_interrupts_and_time_travel 코드 03 (4/4) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 229. 08_interrupts_and_time_travel 코드 04

이제 08_interrupts_and_time_travel 코드 04 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 230. 08_interrupts_and_time_travel 코드 05 (1/2)

이제 08_interrupts_and_time_travel 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 func에서 entrypoint, task를 준비합니다. 데

코레이터 task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 generate_proposal, proposal_workflow는 실제 작업 단위입니다. 핵심 API는 InMemorySaver, invoke, interrupt입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 231. 08_interrupts_and_time_travel 코드 05 (2/2)

이제 08_interrupts_and_time_travel 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 232. 08_interrupts_and_time_travel 코드 06 (1/2)

이제 08_interrupts_and_time_travel 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Literal, TypedDict; graph에서 StateGraph, START, END를 준비합니다. 클래스 ApprovalState는 데이터 구조나 정책 단위를 정의합니다. 함수 approval_node, proceed, cancel는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, interrupt, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 233. 08_interrupts_and_time_travel 코드 06 (2/2)

이제 08_interrupts_and_time_travel 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 234. 08_interrupts_and_time_travel 코드 07 (1/2)

이제 08_interrupts_and_time_travel 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 클래스 ReviewEditState는 데이터 구조나 정책 단위를 정의합니다. 함수 generate, review_edit는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, compile, add_node, add_edge, interrupt입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 235. 08_interrupts_and_time_travel 코드 07 (2/2)

이제 08_interrupts_and_time_travel 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 236. 08_interrupts_and_time_travel 코드 08 (1/3)

이제 08_interrupts_and_time_travel 코드 08 (1/3) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 send_email는 실제 작업 단위입니다. 핵심 API는 interrupt, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 237. 08_interrupts_and_time_travel 코드 08 (2/3)

이제 08_interrupts_and_time_travel 코드 08 (2/3) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 클래스 ToolState는 데이터 구조나 정책 단위를 정의합니다. 함수 email_node는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 238. 08_interrupts_and_time_travel 코드 08 (3/3)

이제 08_interrupts_and_time_travel 코드 08 (3/3) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 239. 08_interrupts_and_time_travel 코드 09 (1/2)

이제 08_interrupts_and_time_travel 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 클래스 AgeState는 데이터 구조나 정책 단위를 정의합니다. 함수 get_age_node는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설

정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 240. 08_interrupts_and_time_travel 코드 09 (2/2)

이제 08_interrupts_and_time_travel 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 241. 08_interrupts_and_time_travel 코드 10 (1/2)

이제 08_interrupts_and_time_travel 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; messages에서 HumanMessage를 준비합니다. 함수 chatbot는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 242. 08_interrupts_and_time_travel 코드 10 (2/2)

이제 08_interrupts_and_time_travel 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 243. 08_interrupts_and_time_travel 코드 11 (1/2)

이제 08_interrupts_and_time_travel 코드 11 (1/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 TypedDict; graph에서 StateGraph, START, END를 준비합니다. 클래스 JokeState는 데이터 구조나 정책 단위를 정의합니다. 함수 generate_topic, write_joke는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, invoke, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 244. 08_interrupts_and_time_travel 코드 11 (2/2)

이제 08_interrupts_and_time_travel 코드 11 (2/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemorySaver, compile, invoke, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 245. 08_interrupts_and_time_travel 코드 12

이제 08_interrupts_and_time_travel 코드 12 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 246. 08_interrupts_and_time_travel 코드 13

이제 08_interrupts_and_time_travel 코드 13 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 29이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 messages에서 AIMessage를 준비합니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 247. 08_interrupts_and_time_travel 코드 14 (1/2)

이제 08_interrupts_and_time_travel 코드 14 (1/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 31이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; types에서 interrupt, Command를 준비합니다. 클래스 ReviewState는 데이터 구조나 정책 단위를 정의합니다. 함수 draft, review, finalize는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver, compile, add_node, add_edge, interrupt입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 248. 08_interrupts_and_time_travel 코드 14 (2/2)

이제 08_interrupts_and_time_travel 코드 14 (2/2) 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 31이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵

심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 249. 08_interrupts_and_time_travel 코드 15

이제 08_interrupts_and_time_travel 코드 15 슬라이드입니다. 노트북 위치는 08_interrupts_and_time_travel.ipynb의 cell 32이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 250. 09_subgraphs.ipynb

이제 CODE 09_subgraphs.ipynb로 넘어갑니다. 09_subgraphs.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/09_subgraphs.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

### 251. 09_subgraphs 코드 01

이제 09_subgraphs 코드 01 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 252. 09_subgraphs 코드 02

이제 09_subgraphs 코드 02 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 253. 09_subgraphs 코드 03 (1/2)

이제 09_subgraphs 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; typing에서 TypedDict, Annotated를 준비합니다. 클래스 AnalysisState는 데이터 구조나 정책 단위를 정의합니다. 함수 count_words, count_chars는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 254. 09_subgraphs 코드 03 (2/2)

이제 09_subgraphs 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 255. 09_subgraphs 코드 04 (1/2)

이제 09_subgraphs 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 클래스 PipelineState는 데이터 구조나 정책 단위를 정의합니다. 함수 create_summary는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 256. 09_subgraphs 코드 04 (2/2)

이제 09_subgraphs 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 257. 09_subgraphs 코드 05 (1/7)

이제 09_subgraphs 코드 05 (1/7) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 7장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 TypedDict; graph에서 StateGraph, START, END를 준비합니다. 클래스 SubState는 데이터 구조나 정책 단위를 정의합니다. 함수 analyze_sentiment는 실제 작업 단위입니다. 핵심 API는 StateGraph입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 258. 09_subgraphs 코드 05 (2/7)

이제 09_subgraphs 코드 05 (2/7) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 259. 09_subgraphs 코드 05 (3/7)

이제 09_subgraphs 코드 05 (3/7) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 클래스 ParentState는 데이터 구조나 정책 단위를 정의합니다. 핵심 API는 compile, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 260. 09_subgraphs 코드 05 (4/7)

이제 09_subgraphs 코드 05 (4/7) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 call_sentiment_subgraph는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 261. 09_subgraphs 코드 05 (5/7)

이제 09_subgraphs 코드 05 (5/7) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 5번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 generate_report는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 262. 09_subgraphs 코드 05 (6/7)

이제 09_subgraphs 코드 05 (6/7) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 6번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 263. 09_subgraphs 코드 05 (7/7)

이제 09_subgraphs 코드 05 (7/7) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 7번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 264. 09_subgraphs 코드 06 (1/3)

이제 09_subgraphs 코드 06 (1/3) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 messages에서 HumanMessage, AnyMessage; graph에서 MessagesState를 준비합니다. 함수 translator, summarizer는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 265. 09_subgraphs 코드 06 (2/3)

이제 09_subgraphs 코드 06 (2/3) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 266. 09_subgraphs 코드 06 (3/3)

이제 09_subgraphs 코드 06 (3/3) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 267. 09_subgraphs 코드 07 (1/2)

이제 09_subgraphs 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 268. 09_subgraphs 코드 07 (2/2)

이제 09_subgraphs 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 269. 09_subgraphs 코드 08 (1/3)

이제 09_subgraphs 코드 08 (1/3) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent;

tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 fruit_info는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 270. 09_subgraphs 코드 08 (2/3)

이제 09_subgraphs 코드 08 (2/3) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ToolCallLimitMiddleware를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 ask_fruit_expert는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke, interrupt입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 271. 09_subgraphs 코드 08 (3/3)

이제 09_subgraphs 코드 08 (3/3) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 272. 09_subgraphs 코드 09 (1/2)

이제 09_subgraphs 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 MessagesState, StateGraph; agents에서 create_agent를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 veggie_info, create_sub_agent는 실제 작업 단위입니다. 핵심 API는 create_agent, StateGraph, compile, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 273. 09_subgraphs 코드 09 (2/2)

이제 09_subgraphs 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 09_subgraphs.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 274. 10_production.ipynb

이제 CODE 10_production.ipynb로 넘어갑니다. 10_production.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/10_production.ipynb입니다. 슬라이드만 넘기지 말고 같

은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

### 275. 10_production 코드 01

이제 10_production 코드 01 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 276. 10_production 코드 02

이제 10_production 코드 02 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 277. 10_production 코드 03

이제 10_production 코드 03 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 json를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 278. 10_production 코드 04 (1/3)

이제 10_production 코드 04 (1/3) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; typing에서 TypedDict를 준비합니다. 클래스 TestState는 데이터 구조나 정책 단위를 정의합니다. 함수 process는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 279. 10_production 코드 04 (2/3)

이제 10_production 코드 04 (2/3) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 test_process, test_empty_input는 실제 작업 단위입니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 280. 10_production 코드 04 (3/3)

이제 10_production 코드 04 (3/3) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 281. 10_production 코드 05 (1/2)

이제 10_production 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 language_models에서 GenericFakeChatModel; messages에서 AIMessage, HumanMessage, AnyMessage를 준비합니다. 함수 chatbot는 실제 작업 단위입니다. 핵심 API는 StateGraph, invoke, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 282. 10_production 코드 05 (2/2)

이제 10_production 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 283. 10_production 코드 06

이제 10_production 코드 06 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 284. 11_local_server.ipynb

이제 CODE 11_local_server.ipynb로 넘어갑니다. 11_local_server.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/11_local_server.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

### 285. 11_local_server 코드 01

이제 11_local_server 코드 01 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 286. 11_local_server 코드 02

이제 11_local_server 코드 02 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 287. 11_local_server 코드 03

이제 11_local_server 코드 03 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 288. 11_local_server 코드 04

이제 11_local_server 코드 04 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 289. 11_local_server 코드 05

이제 11_local_server 코드 05 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 json를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 290. 11_local_server 코드 06 (1/2)

이제 11_local_server 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저

import는 asyncio를 준비합니다. 함수 main는 실제 작업 단위입니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 291. 11_local_server 코드 06 (2/2)

이제 11_local_server 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 292. 11_local_server 코드 07

이제 11_local_server 코드 07 슬라이드입니다. 노트북 위치는 11_local_server.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 293. 12_durable_execution.ipynb

이제 CODE 12_durable_execution.ipynb로 넘어갑니다. 12_durable_execution.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/12_durable_execution.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

### 294. 12_durable_execution 코드 01

이제 12_durable_execution 코드 01 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

### 295. 12_durable_execution 코드 02

이제 12_durable_execution 코드 02 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 296. 12_durable_execution 코드 03

이제 12_durable_execution 코드 03 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 call_api는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 297. 12_durable_execution 코드 04

이제 12_durable_execution 코드 04 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 func에서 task를 준비합니다. 데코레이터 task가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 _make_request, call_api는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 298. 12_durable_execution 코드 05 (1/2)

이제 12_durable_execution 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 TypedDict; graph에서 StateGraph, START, END를 준비합니다. 클래스 DocState는 데이터 구조나 정책 단위를 정의합니다. 함수 write_draft, finalize는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 299. 12_durable_execution 코드 05 (2/2)

이제 12_durable_execution 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 300. 12_durable_execution 코드 06 (1/2)

이제 12_durable_execution 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 func에서 entrypoint, task; memory에서 InMemorySaver를 준비합니다. 데코레이터 task, task, entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 generate_draft, review_draft, write_document는 실제 작업 단위입니다. 핵심 API는 InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 301. 12_durable_execution 코드 06 (2/2)

이제 12_durable_execution 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 302. 12_durable_execution 코드 07 (1/3)

이제 12_durable_execution 코드 07 (1/3) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 TypedDict; graph에서 StateGraph, START, END를 준비합니다. 클래스 PipelineState는 데이터 구조나 정책 단위를 정의합니다. 함수 step_one, step_two는 실제 작업 단위입니다. 핵심 API는 StateGraph, InMemorySaver입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 303. 12_durable_execution 코드 07 (2/3)

이제 12_durable_execution 코드 07 (2/3) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, InMemorySaver, compile, invoke, add_node, add_edge입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 304. 12_durable_execution 코드 07 (3/3)

이제 12_durable_execution 코드 07 (3/3) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 305. 12_durable_execution 코드 08 (1/2)

이제 12_durable_execution 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 metadata에서 version; errors에서 NodeError를 준비합니다. 함수 payment_error_handler는 실제 작업 단위입니다. 핵심 API는 add_node, Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 306. 12_durable_execution 코드 08 (2/2)

이제 12_durable_execution 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 long_running_node는 실제 작업 단위입니다. 핵심 API는 add_node입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 307. 12_durable_execution 코드 09 (1/2)

이제 12_durable_execution 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 runtime에서 RunControl, Runtime; errors에서 GraphDrained를 준비합니다. 함수 my_node는 실제 작업 단위입니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 308. 12_durable_execution 코드 09 (2/2)

이제 12_durable_execution 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 309. 12_durable_execution 코드 10

이제 12_durable_execution 코드 10 슬라이드입니다. 노트북 위치는 12_durable_execution.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing_extensions에서 Annotated, TypedDict; delta에서 DeltaChannel를 준비합니다. 클래스 State는 데이터 구조나 정책 단위를 정의합니다. 함수 append는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 310. 13_api_guide_and_pregel.ipynb

이제 CODE 13_api_guide_and_pregel.ipynb로 넘어갑니다. 13_api_guide_and_pregel.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 03_langgraph/13_api_guide_and_pregel.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

### 311. 13_api_guide_and_pregel 코드 01

이제 13_api_guide_and_pregel 코드 01 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 312. 13_api_guide_and_pregel 코드 02

이제 13_api_guide_and_pregel 코드 02 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 313. 13_api_guide_and_pregel 코드 03 (1/2)

이제 13_api_guide_and_pregel 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 TypedDict; constants에서 START를 준비합니다. 클래스 Essay는 데이터 구조나 정책 단위를 정의합니다. 함수 write_essay, score_essay는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 314. 13_api_guide_and_pregel 코드 03 (2/2)

이제 13_api_guide_and_pregel 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, invoke, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 315. 13_api_guide_and_pregel 코드 04 (1/2)

이제 13_api_guide_and_pregel 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 TypedDict; func에서 entrypoint, task를 준비합니다. 클래스 EssayResult는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 task, task가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 write_essay_func, score_essay_func는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 316. 13_api_guide_and_pregel 코드 04 (2/2)

이제 13_api_guide_and_pregel 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 데코레이터 entrypoint가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 essay_pipeline는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 317. 13_api_guide_and_pregel 코드 05 (1/2)

이제 13_api_guide_and_pregel 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 channels에서 EphemeralValue; pregel에서 Pregel, NodeBuilder를 준비합니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 318. 13_api_guide_and_pregel 코드 05 (2/2)

이제 13_api_guide_and_pregel 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 319. 13_api_guide_and_pregel 코드 06 (1/4)

이제 13_api_guide_and_pregel 코드 06 (1/4) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 4장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 pregel에서 Pregel, NodeBuilder를 준비합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

### 320. 13_api_guide_and_pregel 코드 06 (2/4)

이제 13_api_guide_and_pregel 코드 06 (2/4) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 321. 13_api_guide_and_pregel 코드 06 (3/4)

이제 13_api_guide_and_pregel 코드 06 (3/4) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 reducer는 실제 작업 단위입니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 322. 13_api_guide_and_pregel 코드 06 (4/4)

이제 13_api_guide_and_pregel 코드 06 (4/4) 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 323. 13_api_guide_and_pregel 코드 07

이제 13_api_guide_and_pregel 코드 07 슬라이드입니다. 노트북 위치는 13_api_guide_and_pregel.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Annotated, Sequence; typing_extensions에서 TypedDict를 준비합니다. 클래스 State는 데이터 구조나 정책 단위를 정의합니다. 함수 list_reducer는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

## 324. LangGraph 심화는 '그래프를 그리는 법'보다 '실행을 통제하는 법'입니다

LangGraph 심화는 '그래프를 그리는 법'보다 '실행을 통제하는 법'입니다. 다음 Deep Agents 실전에서는 이 제어 모델 위에 파일, 서브에이전트, 메모리, 스킬과 선택형 TodoListMiddleware를 얹은 장기 작업 하네스를 올립니다. 이 트랙에서 남겨야 할 핵심은 에이전트 실행을 상태, 노드, 엣지, 체크포인트, interrupt, time travel로 직접 통제하는 일입니다. 실습 코드를 모두 외울 필요는 없습니다. 대신 어떤 상황에서 어떤 레이어를 선택하고, 상태와 도구와 권한을 어디에서 관리할지 말로 설명하는 수준까지 가야 합니다. 이 기준으로 다시 노트북을 열어 보면, 각 코드 셀이 단순 예제가 아니라 운영 가능한 에이전트를 구성하는 하나의 결정 지점으로 보입니다.